

Memoria de prácticas. Extensiones de Machine Learning.

Alumno: Juan Andrés Álvarez Cascales

1. Metodología empleada

Para llevar a cabo esta práctica he empleado el código proporcionado por el profesorado, en los archivos "*Selección de atributos evolutiva para clasificación.ipynb*" y "*Particle swarm optimization para selección de atributos - Representación binaria - Sigmoide.ipynb*" sin necesidad de adaptarlos. La tarea se puede desglosar en tres partes:

i). Primera parte: Aplicar Grid Search para encontrar la mejor configuración de hiperparámetros. Para ello la estrategia que llevé a cabo fue simplemente emplear bucles anidados para poder iterar por los distintos valores de los hiperparámetros, de tal forma que se pudieran probar distintas configuraciones.

Para reducir el efecto de la aleatoridad, y estar seguro de que el desempeño obtenido por cada configuración de hiperparámetros no era fruto del azar, ejecuté tres veces cada configuración, cada una de ellas con distinta semilla. De esta forma, para determinar qué configuración es la más idónea, selecciono aquella que reporta el mejor valor promedio de fitness entre las 3 ejecuciones.

Para esta primera parte de búsqueda de hiperparámetros, solo empleo los datos de train.

ii). Segunda Parte: Una vez encontrados los valores óptimos de hiperparámetros, ejecutamos el algoritmo de selección de atributos (wrapped), haciendo uso de dicha configuración óptima. Lo ejecutamos 10 veces, cada una de ellas con distinta semilla.

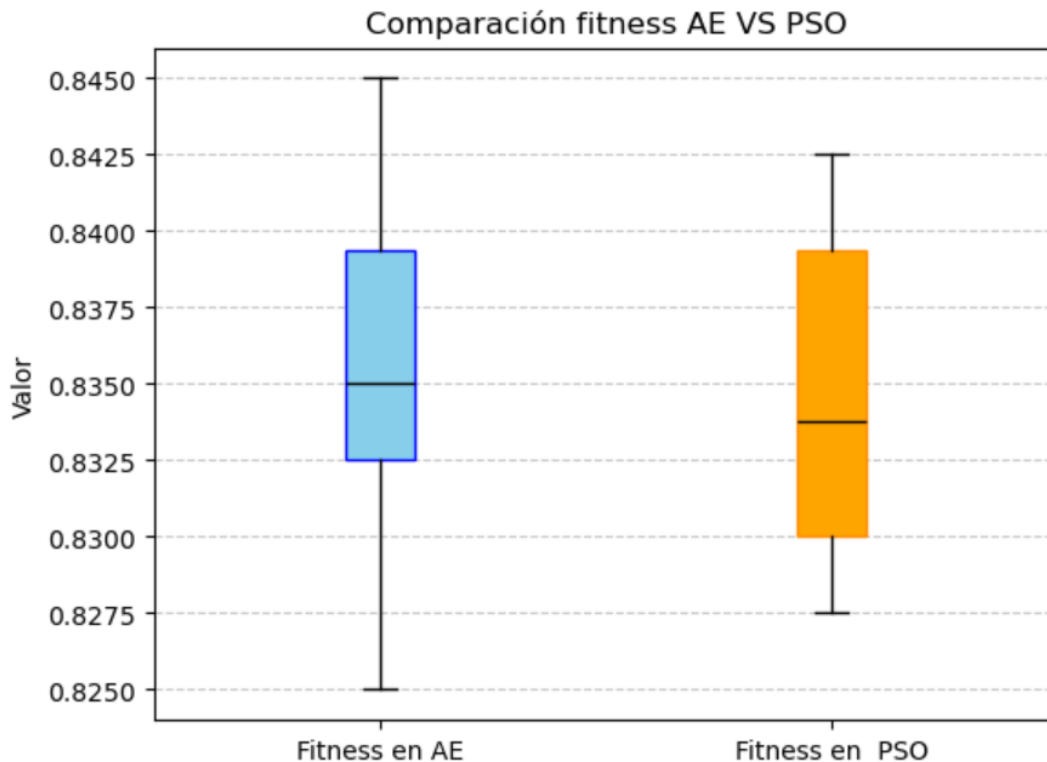
Al estar haciendo *feature selection* 10 veces con distinta semilla, a pesar de usar en todas ellas la misma configuración de hiperparámetros, nos reporta 10 subconjuntos de atributos distintos.

Al igual que en la primera parte, solo se emplean los datos de entrenamiento. La métrica de adecuación del modelo se obtiene mediante validación cruzada con 5 folds sobre el conjunto de datos de entrenamiento.

iii). Tercera Parte: Cada uno de los subconjuntos de atributos que obtenemos como solución tras las ejecuciones de los algoritmos bioinspirados en la parte anterior, son empleados para entrenar un modelo RandomForest. Así tenemos 10 modelos entrenados, pero con distintos datos. Por último, evalúo dichos modelos con los datos de test guardando su accuracy para su posterior análisis.

2. Análisis de los resultados y Conclusiones

2.1 Comparación resultados fitness



Atendiendo a la comparación entre los valores de fitness obtenidos mediante el algoritmo evolutivo (AE) y el algoritmo de optimización por enjambre de partículas (PSO), se observa que el AE alcanza una mediana ligeramente superior a la del PSO, lo que indica que, en promedio, las soluciones (subconjuntos de atributos) generadas por el AE presentan un mejor rendimiento.

Además, las soluciones seleccionadas por AE tienen una mayor dispersión en los resultados, con un rango intercuartílico y valores extremos más amplios, lo que sugiere una mayor variabilidad entre ejecuciones y, por tanto, una menor estabilidad en el proceso de optimización.

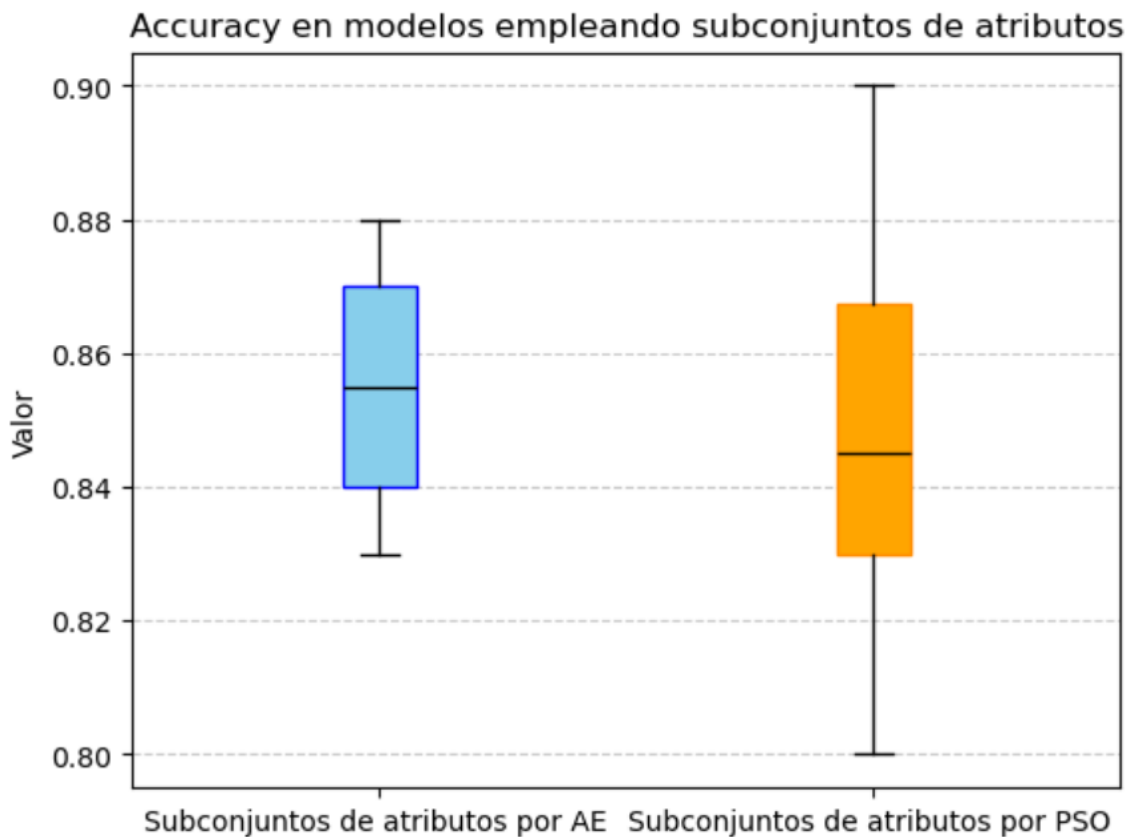
En contraste, las soluciones obtenidas por PSO presentan resultados más consistentes, con una menor variabilidad, y con una mediana de fitness ligeramente inferior. Lo cual nos indica que para estos datos de train, las soluciones óptimas encontradas por PSO, se comportan algo peor, pero obteniendo una adecuación (fitness, métrica de validación) menos dispersa.

Esto nos indica que el AE explora con mayor amplitud el espacio de búsqueda, pudiendo alcanzar soluciones potencialmente mejores, pero más dependientes de la aleatoriedad, mientras que el PSO ofrece un comportamiento más estable y reproducible, aunque con un rendimiento medio ligeramente menor.

Esto tiene sentido ya que el AE genera los individuos de la población de forma aleatoria buscando cubrir uniformemente el espacio de búsqueda, lo que puede derivar en soluciones más variables y dependientes de la aleatoriedad que las obtenidas por PSO, que, a pesar de ser generadas también aleatoriamente, se mueven por el espacio de

búsqueda, guiadas por su experiencia y la del grupo, lo que enfoca las soluciones hacia espacios más recortados.

2.2 Comparación Accuracy modelos entrenados con las selecciones de atributos

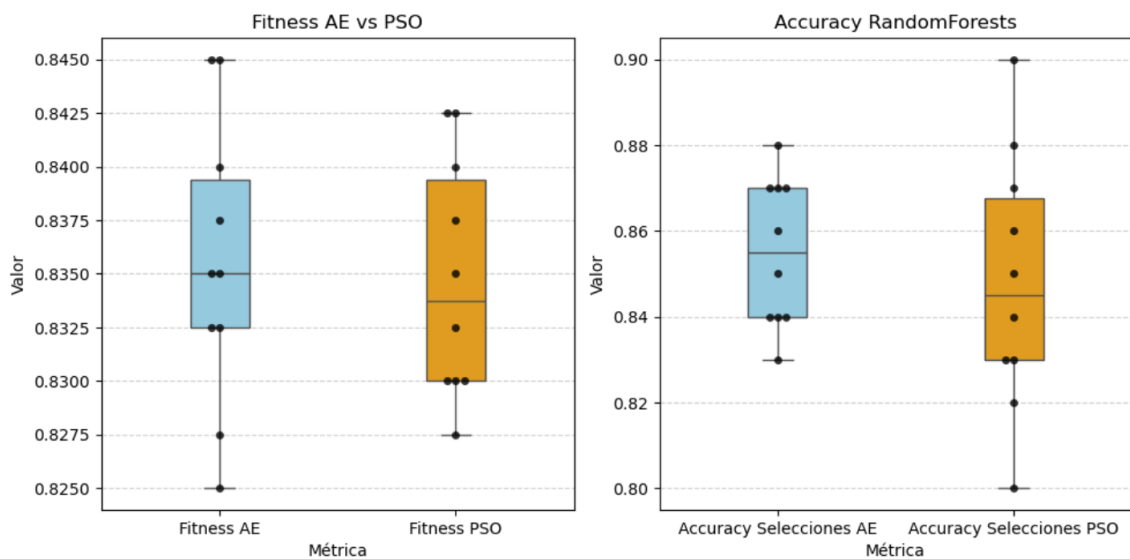


En esta segunda gráfica se comparan los valores de accuracy en el conjunto de prueba obtenidos por modelos Random Forest entrenados utilizando las distintas selecciones de atributos generadas por el algoritmo evolutivo (AE) y el de optimización por enjambre de partículas (PSO). Cada modelo se entrenó con una semilla diferente y empleando un subconjunto de atributos distinto, de modo que los resultados reflejan la capacidad de generalización de las soluciones de *feature selection* obtenidas por cada método.

Se observa que los subconjuntos de atributos generados por AE presentan una mediana de accuracy ligeramente superior frente a los del PSO, lo que concuerda con la adecuación de las soluciones de la gráfica anterior.

Cabe destacar que, el rango intercuartílico del accuracy de los subconjuntos obtenidos por AE es más estrecho, lo que indica una menor variabilidad y mayor consistencia entre ejecuciones, para estos datos de test. En cambio, los subconjuntos de atributos seleccionados por PSO muestran una dispersión más amplia, aunque alcanzan el valor máximo más alto (0.90), también tienen mínimos sensiblemente menores, reflejando un comportamiento menos estable.

2.3 Conclusión Final

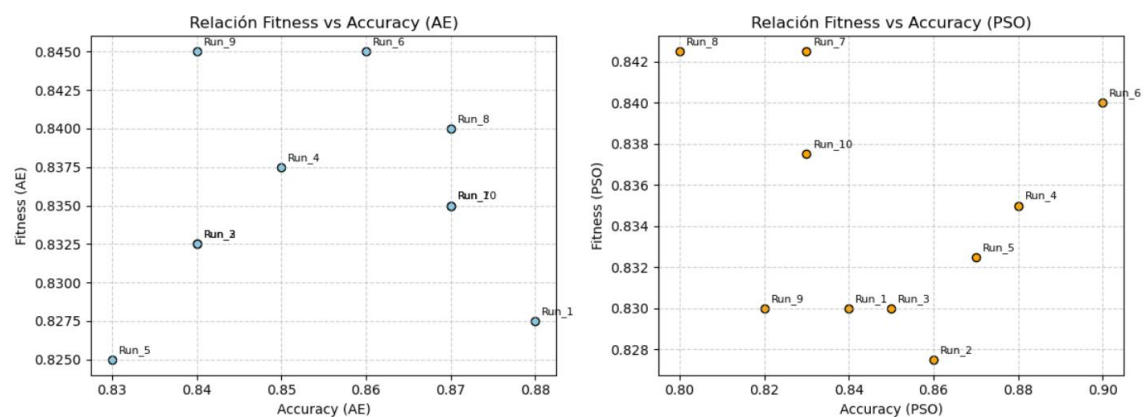


Véase como las soluciones obtenidas por los algoritmos AE y PSO atendiendo a solamente los datos de entrenamiento, reflejaban una mayor dispersión del fitness para el AE, frente a una menor variabilidad en PSO, sin embargo, al validar dichas soluciones con los datos de test, vemos que ocurre lo opuesto, las soluciones generadas por AE pasan a reportar un accuracy más consistente, frente a las generadas por PSO, que pasan a tener mayor variabilidad.

Esto nos está indicando que las soluciones (selecciones de atributos) obtenidas mediante AE tienden a generalizar mejor, manteniendo un rendimiento alto y consistente en datos no vistos, mientras que las obtenidas mediante PSO, aunque pueden llegar a ofrecer los mejores resultados individuales, presentan una menor robustez global.

Dado que la estimación de los mejores conjuntos de atributos se llevó a cabo solo con datos de train, estos resultados de la segunda gráfica también podrían indicar que en los datos de test existen patrones o relaciones entre atributos que no estaban presentes o eran menos evidentes en el conjunto de entrenamiento, afectando de manera diferente la capacidad de generalización de las soluciones obtenidas por estos algoritmos metaheurísticos bioinspirados.

Comparación: Fitness vs Accuracy para AE y PSO



En estos diagramas de dispersión por algoritmo bioinspirado, que comparan los datos de fitness y accuracy, podemos ver como en la gráfica del AE hay una cierta tendencia a que las soluciones más prometedoras (conjuntos de atributos seleccionados solo en train) cuanto mayor es su fitness, mejor desempeño tienen generalizando para datos nuevos en test. Sin embargo, en la gráfica de PSO se aprecia como las soluciones más prometedoras no tienen por qué tener igualmente un desempeño tan bueno en datos de test. (No quiere decir que la generalización de las soluciones obtenidas por PSO no sea buena, sino que no todas suelen ser tan buenas generalizando para datos nuevos)

3. Anexo Prácticas Formativas:

Durante la realización de las prácticas formativas encontré un par de resultados curiosos que vale la pena exponer.

En la [práctica 2](#), “Ajuste evolutivo de hiperparámetros” el modelo subyacente bajo el algoritmo evolutivo es un MLPClassifier. En esta tarea se pretende averiguar cuál es la configuración de hiperparámetros que reporta un mejor fitness, usando un Algoritmo evolutivo.

En mi caso, se consigue obtener un accuracy de 0.905, con 32 neuronas (una sola capa), la función de activación reLU, un learning rate del tipo invscaling (se hace progresivamente más pequeño con el avance de las iteraciones, favoreciendo a la estabilidad de convergencia al acercarse al mínimo), y unos valores de $\beta_1 = 0.26$ y $\beta_2 = 0.059246$.

```
ACC cv: 0.905  
Solution: beta_1: 0.261358 beta_2: 0.059243 hidden_layer_size: 32 activation: relu learning_rate: invscaling
```

Lo inusual en este caso es el valor tan bajo de los valores beta. Dichos parámetros son los encargados de controlar el **momento** al usar el optimizador adam (el que usa por defecto). Aunque el valor de los betas se mueve en el espacio $[0,1)$, lo normal es que tengan valores muy cercanos a 1, en torno a 0.9 y 0.99 respectivamente.

Cuanto más pequeño sea β_1 , menos peso se le da al histórico anterior de la dirección del gradiente, mientras que cuanto menor es β_2 menos intervención tiene el histórico de los gradientes al cuadrado, por lo que los valores bajos de los betas dan lugar a que el algoritmo adam sea menos estable, con cambios más bruscos. Es decir, el momento, que es la característica principal de este optimizador ADAM, al ser los betas tan bajos, no está realmente entrando en juego.

A pesar de que el Adam no se está comportando como tal, se consiguen valores bastante buenos de accuracy, por lo que para los datos con los que trabajamos y los hiperparámetros encontrados, el modelo está teniendo un muy buen rendimiento. Tiene mejor desempeño que al usar los hiperparámetros por defecto.

Que el algoritmo evolutivo haya encontrado estos valores para los betas se debe a no haber aplicado limitaciones más restrictivas al espacio de búsqueda, estableciendo solo que los valores deben estar entre 0 y 1 (la inicialización de los individuos se hace de forma aleatoria y uniforme en todo el espacio de búsqueda de las variables). Aunque β_1 y β_2 matemáticamente pueden estar en $[0, 1)$, en la práctica los valores útiles están en

intervalos muy reducidos, al dejar el rango completo, el algoritmo consigue llegar a valores en los que el adam deja de comportarse realmente como adam.

También es cierto que en nuestra función de evaluación no comprobamos la falta de convergencia del optimizador, un modelo con betas bajos puede no converger o hacerlo rápidamente a una solución puntual cualquiera, sin poder llegar a converger en el óptimo. Por lo que, si convergió a una solución que por casualidad dió lugar a un buen valor del accuracy, el algoritmo evolutivo lo premió. Por lo tanto, puede ser que el Adam no haya convergido, pero la solución a la que haya llegado si nos esté dando muy buenos resultados.

En la práctica 3, “Particle swarm optimization para selección de atributos - Representación binaria - Sigmoide”, el resultado obtenido tras la búsqueda de hiperparámetros, es que emplear 13 atributos frente a los 20 totales nos reporta un mejor accuracy (0.835, frente a 0.76), pero esto es solo teniendo en cuenta los datos de train, usados con validación cruzada, sin validar posteriormente con datos de test.

```
Solución del algoritmo (atributos seleccionados)
x: [0 1 1 1 1 0 0 1 1 0 0 1 1 1 0 1 1 1 0 1]
Número de atributos seleccionados: 13
ACC cv con los atributos seleccionados: 0.8350000000000002
```

Solución con todos los atributos

```
n = len(X_train.T)
all_attributes = np.ones(n,dtype='int')
acc = f(all_attributes,X_train,y_train,Phi,cv)
print("Número total de atributos:",n)
print("ACC cv con todos los atributos: ",acc)
```

```
Número total de atributos: 20
ACC cv con todos los atributos: 0.76
```

Al entrenar de nuevo dos modelos, uno con todos los atributos y otro solo con la selección que mejor resultados reportó en la *feature selection*, y validarlos posteriormente con los datos de test, se observó que el clasificador entrenado con la selección de atributos alcanzó un accuracy inferior al del modelo entrenado con todos los atributos, cuando, al evaluar únicamente sobre los datos de entrenamiento mediante validación cruzada, ocurría lo contrario.

```
Clasificador con los atributos seleccionados  
ACC con los datos de test = 0.79  
Clasificador con todos los atributos  
ACC con los datos de test = 0.83
```

Esto puede deberse a que el PSO selecciona los atributos buscando optimizar la función de *fitness*, pero al hacerlo únicamente sobre los datos de entrenamiento, puede haber escogido el conjunto de características que mejor funcionan en ese entorno específico. En consecuencia, el modelo obtiene un buen desempeño en los datos de entrenamiento, pero a costa de estar aprendiendo peculiaridades y ruido de los datos, lo que realmente no le sirva para generalizar.

Es decir, estamos ante un ligero caso de sobreajuste, con la selección de los 13 atributos el modelo ha encontrado un subconjunto que funciona bien en train, pero que realmente no generaliza tan bien con otros datos nuevos.